APPLIED DATA SCIENCE · LECTURE SERIES

Demand Forecasting & Inventory Optimization

From raw retail sales to a stocking decision: a complete, reproducible CRISP-DM case study on the Walmart M5 dataset

PREPARED BY

Haidara Mohammad

Machine Learning and AI Engineer

WHAT YOU WILL BE ABLE TO DO AFTER THESE NOTES

- Frame a forecasting problem in business terms and structure it with CRISP-DM.
- Clean and validate a large panel of daily sales without leaking the future.
- Decompose a series, diagnose seasonality and classify intermittent demand.
- Build and explain benchmarks, ETS, SARIMA, Croston-type and LightGBM models.
- Evaluate forecasts honestly with rolling-origin backtesting and WRMSSE.
- Combine models into ensembles and explain why that works.
- Turn a forecast into safety stock, reorder points and a dollar figure.
- Defend every design choice in a technical interview.

Companion to the GitHub project **retail-demand-forecasting** (package `demandcast`).
Every number in these notes is generated by the project's pipeline and inserted automatically.
Dataset slice: store **CA_3**, department **FOODS_3** · 823 products · 2013-01-01 to 2016-05-22.
Generated 07 November 2026.

Contents

1. The business problem and the CRISP-DM method
2. Data understanding: the M5 dataset
3. Data preparation: from wide CSV to a clean panel
4. Exploratory analysis: trend, seasonality, intermittency
5. How to evaluate a forecast: backtesting and metrics
6. Benchmark models: the bar every model must clear
7. Statistical models: ETS, SARIMA, Croston, ADIDA
8. Machine learning: a global LightGBM model
9. Demand signal enrichment: is external data worth it?
10. Ensembles: why combining forecasts works
11. Results of the model comparison
12. From forecast to decision: inventory optimization
13. Deployment: the production forecast and the inventory plan
14. Engineering the project: structure, tests, CI, reproducibility
15. Presenting the project in an interview
16. Exercises and extensions
17. Glossary and further reading

HOW TO USE THESE NOTES

Read them with the code open. Each lecture names the file that implements the idea (for example `src/demandcast/features.py`). The fastest way to really own this project is to re-derive each number yourself: change one setting in `configs/default.yaml`, rerun the pipeline and explain what changed and why. Coloured boxes mark **key ideas** (blue), **professor's notes** (amber), **common mistakes** (red) and **interview angles** (green).

LECTURE 1

The business problem and the CRISP-DM method

1.1 Why retailers forecast demand

A supermarket stocks tens of thousands of products. For every one of them, someone has to decide *how many units to order and when*. Order too few and the shelf goes empty: the customer buys nothing or buys from a competitor, which is lost revenue and lost goodwill. Order too many and money sits on the shelf: capital is tied up, storage costs accrue, and fresh food expires. Both errors cost money, and they pull in opposite directions.

A demand forecast is the quantity that balances these two costs. It is not an academic exercise: a one-point improvement in forecast accuracy, multiplied across thousands of products and hundreds of stores, is worth millions. That is also why this topic interviews so well: the business value is obvious to any listener.

> **KEY IDEA**
>
> The goal is never "a low error metric". The goal is a better **decision**: fewer empty shelves for the same inventory, or the same availability with less inventory. Our project therefore ends with an inventory simulation that measures exactly that.

1.2 CRISP-DM: a map for data projects

CRISP-DM (Cross-Industry Standard Process for Data Mining) is the most widely used process model for analytics projects. It has six phases, and it is explicitly *iterative*: findings in a later phase often send you back to an earlier one.

Phase	Question it answers	Where in this project
1. Business understanding	What decision are we improving, and how will we measure success?	This lecture; README "Business problem"
2. Data understanding	What data exists, what does it mean, what is wrong with it?	`data.py`, Lectures 2 and 4
3. Data preparation	How do we turn raw files into a modelling table?	`data.py`, `features.py`
4. Modelling	Which techniques, with which settings?	`models.py`, `ensemble.py`
5. Evaluation	Is it accurate, and does it meet the business goal?	`backtest.py`, `evaluation.py`, `inventory.py`
6. Deployment	How does the result reach the people who decide?	`pipeline.py forecast` → inventory plan CSV

1.3 Our concrete problem statement

Business question. For each of the 823 products in department FOODS_3 of Walmart store CA_3, forecast daily unit sales for the next 28 days, and use that forecast to set order quantities that hit a 95% fill rate with the least inventory.

Success criteria. (1) The forecast beats simple benchmarks on WRMSSE across several historical test windows. (2) The inventory policy driven by the forecast needs less stock than one driven by a simple moving average to deliver the same service level.

> **PROFESSOR'S NOTE: WHY 28 DAYS?**
>
> The horizon must match the decision. Stores reorder weekly with a lead time of a few days, so the decision needs about 1–2 weeks of foresight; planners also want a month for staffing and promotions. 28 days is also the horizon of the M5 competition, which lets us compare our setup to published research.

LECTURE 2

Data understanding: the M5 dataset

2.1 Origin

The M-competitions, organised by Spyros Makridakis since 1982, are the reference benchmark of the forecasting field. The fifth edition (M5, 2020) released real Walmart data: daily unit sales of 3,049 products in 10 stores across California, Texas and Wisconsin, about five and a half years of history (January 2011 to June 2016). That makes 30,490 product-store series.

2.2 The three files

File	Content	Shape
`sales_train_evaluation.csv`	One row per product-store, one column per day (`d_1` … `d_1941`): units sold. This is "wide" format.	30,490 × 1,947
`calendar.csv`	One row per day: date, Walmart week id, up to two events (name and type: Sporting, Cultural, National, Religious), and SNAP flags per state.	1,969 × 14
`sell_prices.csv`	Price of each product in each store for each Walmart week. A product only appears once it is on sale.	6.8 M × 4

2.3 What is SNAP?

The Supplemental Nutrition Assistance Program is a U.S. food-assistance benefit. Each state pays it on specific days of the month, and many households shop for food right after receiving it. For a food department this is a strong, known-in-advance demand signal, and it is exactly the kind of "external" information a forecasting model can exploit.

2.4 Why we study one store and one department

All 30,490 series would take hours to model on a laptop with per-series statistical models. We take one slice, store **CA_3** and department **FOODS_3** (the largest department, 823 products), keeping the history from 2013-01-01. The code is fully parameterised: changing two lines of `configs/default.yaml` reruns the entire study on any other store or department. In an interview, say this explicitly: the scope was a compute decision, not a limitation of the method.

> **INTERVIEW ANGLE**
>
> "Why M5 and not a Kaggle toy dataset?" Because it is real transactional data with the problems real data has: products launched mid-history, store closures, price changes, and a majority of products that sell zero units on many days. The data is downloaded from a public mirror, so anyone can reproduce the results without a Kaggle account.

LECTURE 3

Data preparation: from wide CSV to a clean panel

3.1 Wide versus long format

The sales file is *wide*: one column per day. Almost every modelling library wants *long* (also called "tidy") format: one row per observation. We "melt" the table so each row is one product on one day. The forecasting community (Nixtla's libraries, for example) standardises the three key columns as:

```
unique_id   ds           y      sell_price  snap  event_type_1 ...
FOODS_3_001 2013-01-01   0.0    2.50        1     none
FOODS_3_001 2013-01-02   0.0    2.50        1     none
```

`unique_id` identifies the series, `ds` is the timestamp, `y` is the target. A table that stacks many time series like this is called a **panel**.

3.2 Cleaning decisions, and why each matters

Every cleaning step below changes what the model learns. In `data.py` each one is commented with its reason, and the counts are written to `reports/data_quality.json`.

Issue	What we did	Rows affected
Products not yet launched	The sales file shows zeros before a product reaches the shelf. Those are not "zero demand", they are "not available". We drop every row before the product's first listed price.	319,711
Study window	Keep history from 2013-01-01: about 3.4 years, enough to see yearly seasonality three times.	—
Missing prices during a product's life	Forward-fill the last known price (the price did not vanish, the record did).	0
Store closures	Walmart is closed on Christmas Day, so sales are structurally zero. Rows are kept (the calendar must be contiguous) but flagged with `is_closed`.	3 days
No event	Event columns are `NaN` on normal days; we encode them as the category `"none"`.	—

The result: **935,122** rows, 823 series, 37.1% of days with zero sales, on average 4.02 units per product per day.

> **COMMON MISTAKE: TREATING PRE-LAUNCH ZEROS AS DEMAND**
>
> If you keep the zeros before launch, every model learns that the product "used to sell nothing", so its forecasts are biased downward, and the scaling term of RMSSE (Lecture 5) is distorted. This one step is worth mentioning in an interview because it shows you look at what the data *means*, not just at its format.

3.3 Validation: fail loudly

`validate_panel()` asserts four invariants after cleaning: no duplicate (product, day) pairs, no negative sales, no missing prices, and no gaps in the daily calendar of any product. These are cheap checks that catch expensive bugs (for example a bad merge that duplicates rows would silently double every sales figure). The unit test suite also verifies that duplicates are caught.

LECTURE 4

Exploratory analysis: trend, seasonality, intermittency

4.1 The components of a time series

A classical way to think about a series is as the sum of three parts:

$$y_t = T_t + S_t + R_t$$

- **Trend** T: the slow, long-run level (growth or decline of the business).
- **Seasonality** S: a pattern that repeats with a fixed period: day-of-week (period 7) and time-of-year (period 52 weeks or 365 days).
- **Remainder** R: whatever is left: noise, promotions, events, shocks.

If seasonal swings grow with the level of the series, a *multiplicative* form ($y = T \times S \times R$) fits better; taking logarithms turns it into an additive one.

4.2 STL decomposition

STL (Seasonal-Trend decomposition using LOESS, Cleveland et al., 1990) estimates the three components with repeated local regressions. It is robust, handles any period, and lets the seasonal shape change slowly over time. We apply it to **weekly** totals of the department (period = 52), which removes the day-of-week cycle and makes the slower patterns visible.

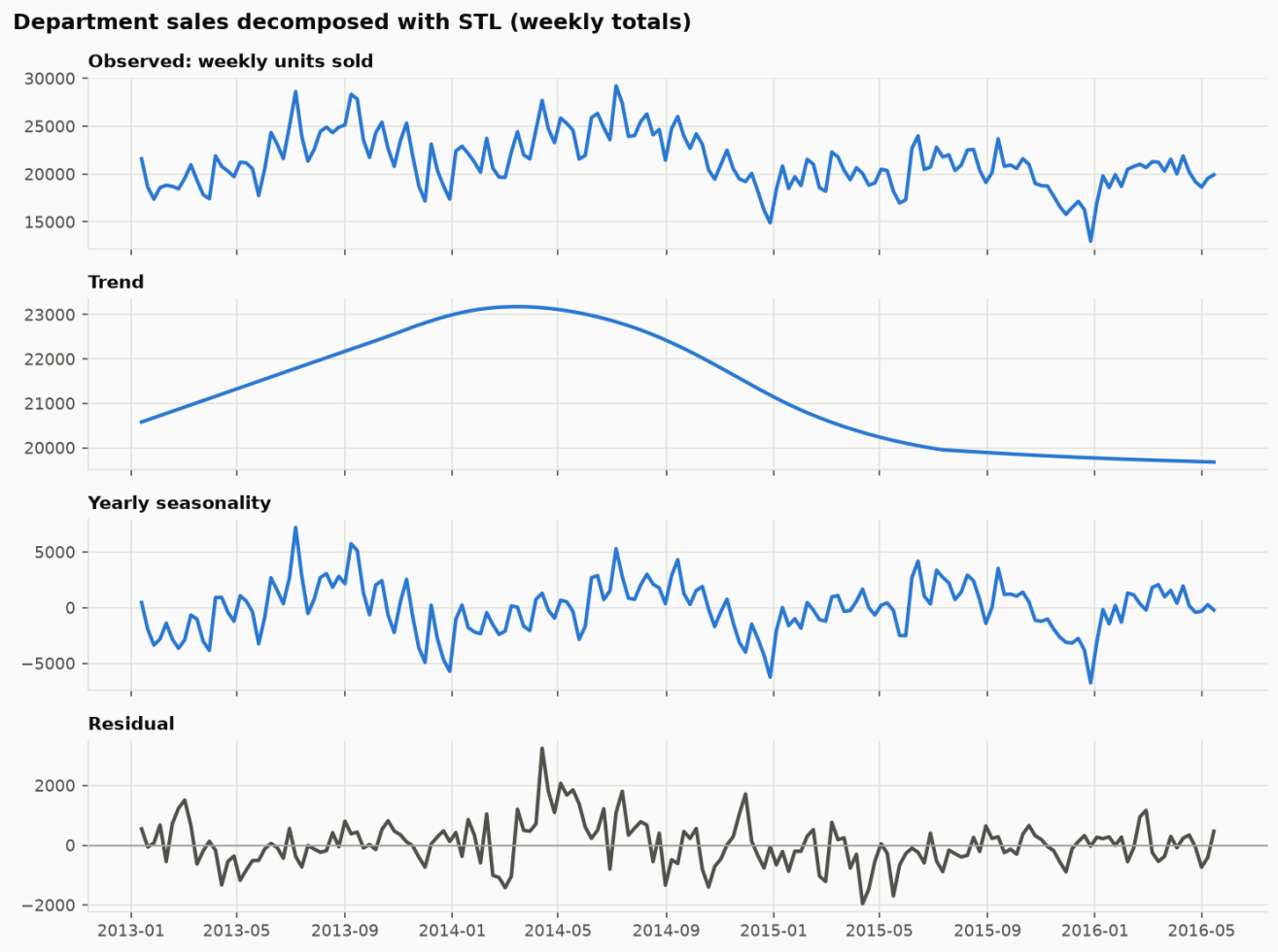


Figure 4.1. STL decomposition of weekly department sales. Demand peaks in summer and dips around the new year; the trend rose until 2014 and declined since. The residual is centred on zero with no remaining structure, a sign the decomposition captured the systematic parts.

4.3 Weekly profile and SNAP effect

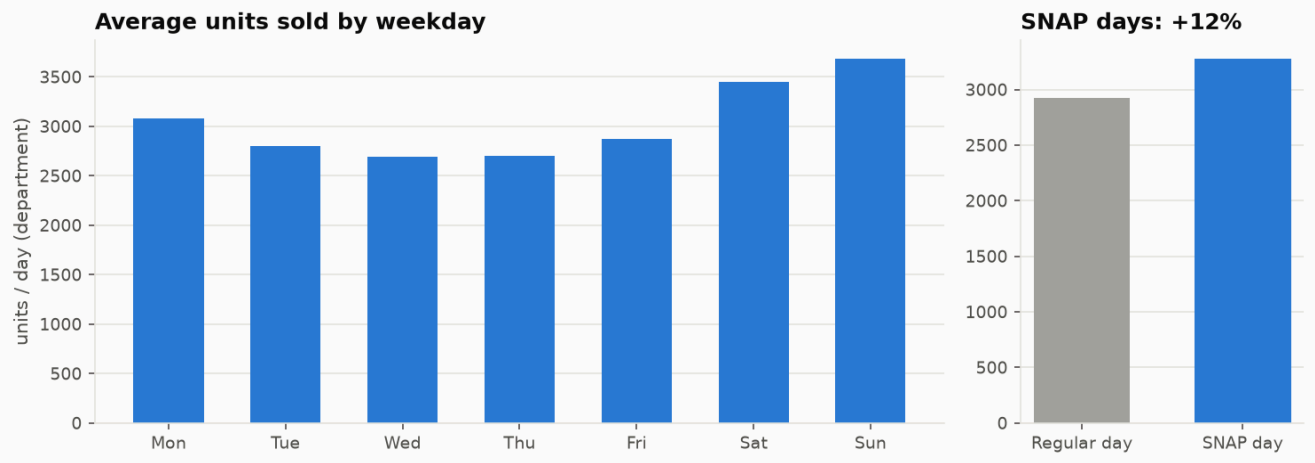


Figure 4.2. Left: weekends sell markedly more than mid-week. Right: on SNAP payment days the department sells 12% more. Both patterns are predictable in advance, so a good model should use them.

4.4 Intermittent demand and the Syntetos-Boylan classification

Aggregated sales look smooth, but individual products often do not. Many sell zero units on most days, then a few units at once. Such **intermittent** series break the assumptions of standard models (which expect continuous, roughly normal values). Syntetos, Boylan and Croston (2005) classify a series with two statistics:

$$\text{ADI} = \frac{\text{number of periods}}{\text{number of periods with non-zero demand}} \qquad \text{CV}^2 = (\sigma / \mu)^2 \text{ of the non-zero demand sizes}$$

ADI ("average demand interval") measures *how often* a product sells; CV^2 measures how *variable* the quantity is when it sells. The cut-offs 1.32 and 0.49 give four classes:

Class	ADI	CV^2	Meaning	SKUs here
Smooth	< 1.32	< 0.49	sells most days, stable sizes	174
Erratic	< 1.32	≥ 0.49	sells most days, sizes jump	87
Intermittent	≥ 1.32	< 0.49	many zero days, stable sizes	351
Lumpy	≥ 1.32	≥ 0.49	many zero days and jumpy sizes: hardest	211

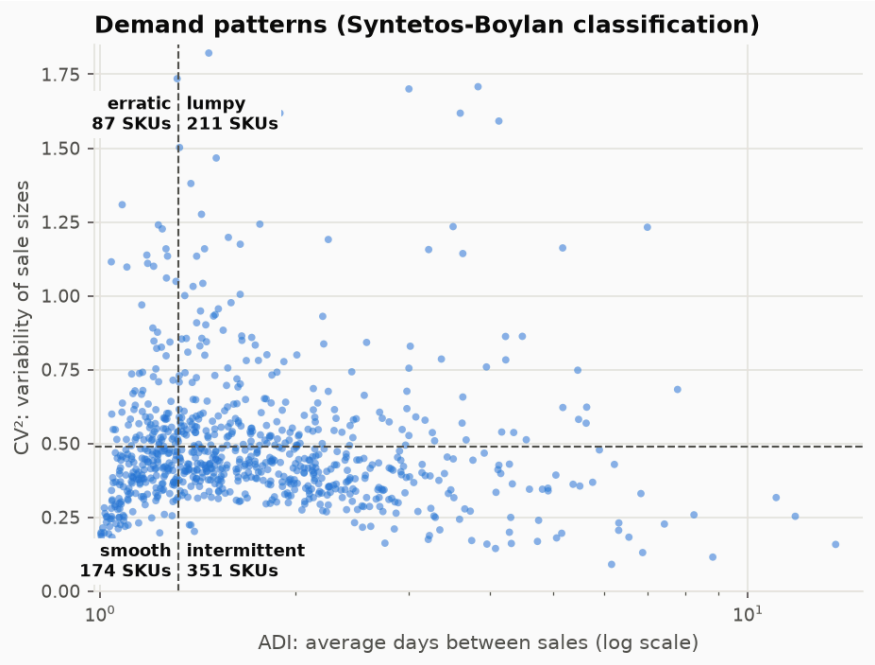


Figure 4.3. Each dot is one product. Most products are intermittent or lumpy, which is why we include Croston-type models designed for such demand (Lecture 7).

> **KEY IDEA: EDA SHOULD CHANGE YOUR MODELLING PLAN**
>
> Each finding here maps to a later design decision: weekly seasonality → season length 7 in every statistical model; SNAP and events → enrichment features; intermittency → Croston/ADIDA models, a Tweedie loss for LightGBM, and a scale-free metric. Explaining that chain is more impressive than any single chart.

LECTURE 5

How to evaluate a forecast: backtesting and metrics

We discuss evaluation *before* models on purpose. If the evaluation is wrong, every comparison that follows is meaningless.

5.1 Why a random train/test split is wrong for time series

In ordinary machine learning we shuffle rows and hold out 20%. For time series this is a disaster: the model would train on Wednesday and Friday to "predict" Thursday, using information that would never be available in real life. That is **data leakage**. The test set must always lie strictly *after* the training data.

5.2 Rolling-origin backtesting

A single time-ordered split is better, but still fragile: one unusual month can make a bad model look good. We therefore repeat the experiment at several **forecast origins** (also called cutoffs). At each origin every model is retrained on all data up to the cutoff and forecasts the next 28 days:

```
fold 1: train ........................ 2016-02-28 | test 28 days
fold 2: train ............................... 2016-03-27 | test 28 days
fold 3: train ..................................... 2016-04-24 | test 28 days
```

This is also called time-series cross-validation. It gives 3 independent out-of-sample results per model, so we can check that the ranking is *stable*, not lucky. See `backtest.py`.

5.3 Leakage beyond the split

Leakage hides in features too. A rolling 7-day mean of sales, computed naively, includes the very day we are predicting. Our rule: **every feature built from sales is lagged by at least the horizon (28 days)**. Prices, calendar, events and SNAP days are allowed because the retailer knows them in advance. The test `test_no_future_leakage` enforces this automatically: it overwrites all sales after the cutoff with 999 and checks that not a single feature in the forecast window changes.

5.4 Point-forecast metrics

Let y be actual sales, $\hat{y}$ the forecast, $e = \hat{y} - y$ the error, over h days.

Metric	Formula	Reads as / caveat
MAE	mean $\lvert e \rvert$	Average units wrong. Scale-dependent: cannot compare a fast and a slow product.
RMSE	$\sqrt{\text{mean } e^2}$	Penalises big misses more. Optimal forecast = the mean.
WAPE	$\Sigma\lvert e \rvert / \Sigma y$	"We are off by X% of volume". Business-friendly, robust to zeros (unlike MAPE).
Bias	$\Sigma e / \Sigma y$	Positive = over-forecasting (excess stock), negative = under-forecasting (stockouts).
RMSSE	$\sqrt{(\text{mean } e^2 / \text{mean } (y_t - y_{t-1})^2_{\text{train}})}$	RMSE divided by the in-sample error of a naive forecast. Scale-free; < 1 means better than naive in-sample.
WRMSSE	$\Sigma_i w_i \cdot \text{RMSSE}_i$	Official M5 metric. w_i = product's share of dollar sales in the last 28 training days.

COMMON MISTAKE: MAPE ON RETAIL DATA

MAPE divides by actual sales. With 37.1% zero-sales days it is undefined or explodes. It also rewards under-forecasting. Use WAPE for communication and a scaled metric (MASE, RMSSE) for model selection.

KEY IDEA: WHY WRMSSE IS THE RIGHT PRIMARY METRIC HERE

Scaled: a product that sells 50 units/day and one that sells 0.3 contribute on the same footing. **Squared**: it targets the mean, which is what inventory planning needs (it sums nicely over the lead time). **Weighted by revenue**: getting the best-sellers right matters more to the business than getting a niche item right.

LECTURE 6

Benchmark models: the bar every model must clear

A sophisticated model that cannot beat a trivial one is worthless, and this happens more often than you would think. Always start with benchmarks.

Model	Forecast for every future day	Intuition
Naive	$\hat{y}_{T+k} = y_T$	"Tomorrow looks like today." Optimal for a random walk.
Seasonal naive	$\hat{y}_{T+k} = y_{T+k-7\cdot\lceil k/7 \rceil}$	"Next Tuesday looks like last Tuesday." Captures weekly seasonality for free. Our reference for "skill".
Moving average (28d)	$\hat{y}_{T+k} = \text{mean}(y_{T-27} \dots y_T)$	"Average of the last month." What many small retailers actually use, so it is the baseline for the inventory comparison.

Forecast skill expresses improvement relative to a benchmark:

$$\text{Skill} = 1 - \text{WRMSSE}_{\text{model}} / \text{WRMSSE}_{\text{seasonal naive}}$$

A skill of 0.20 means the model's error is 20% lower than the seasonal naive's error.

> **PROFESSOR'S NOTE**
>
> Notice that the naive forecast is dreadful for intermittent products: if yesterday was a zero, it forecasts zero for a month; if yesterday had a spike, it forecasts that spike for a month. Smoothing (averaging) is therefore essential for retail SKUs, a lesson that carries over to every model that follows.

LECTURE 7

Statistical models: ETS, SARIMA, Croston, ADIDA

"Statistical" or "local" models fit **one model per product**, using only that product's own history. We use Nixtla's `statsforecast`, which compiles models with numba and parallelises across products. Each statistical model sees the last 730 days before the cutoff (`models.py`).

7.1 Exponential smoothing (ETS)

Simple exponential smoothing keeps a running *level* that is updated as a weighted average of the newest observation and the previous level:

$$\ell_t = \alpha \cdot y_t + (1-\alpha)\cdot \ell_{t-1}, \quad 0 < \alpha < 1$$

Unrolling the recursion shows that weights on past observations decay geometrically: α, $\alpha(1-\alpha)$, $\alpha(1-\alpha)^2$, … Recent data matters most. Holt added a *trend* equation and Winters a *seasonal* one. The modern formulation, **ETS** (Error, Trend, Seasonality; Hyndman et al.), writes these as state-space models where each component can be None, Additive or Multiplicative (trend can also be damped). `AutoETS` fits every admissible combination by maximum likelihood and picks the one with the lowest **AIC** (Akaike Information Criterion = goodness of fit penalised by the number of parameters, so we do not over-fit).

7.2 ARIMA and SARIMA

ARIMA(p, d, q) models a series through its own past values and past errors:

- **AR(p)** autoregression: y_t depends on $y_{t-1} \dots y_{t-p}$.
- **I(d)** integration: difference the series d times ($y_t - y_{t-1}$) to make it stationary (constant mean and variance).
- **MA(q)** moving average: y_t depends on the last q forecast errors.

SARIMA$(p,d,q)(P,D,Q)_m$ adds the same three terms at the seasonal lag m (here $m = 7$): e.g. this Monday depends on last Monday. `AutoARIMA` (Hyndman-Khandakar algorithm) chooses d with unit-root tests, then searches orders stepwise by AIC.

> **PROFESSOR'S NOTE: AN ENGINEERING TRADE-OFF YOU MADE**
>
> The default AutoARIMA search cost about 4.5 seconds per product on the development laptop: about 10 minutes per fold. We constrained the search ($p, q \le 2$; $P, Q \le 1$; approximate likelihood during search), which cut the cost about tenfold. Daily retail series rarely need high orders, so the accuracy cost is negligible. Profiling before optimising, and documenting the trade-off in the code, is precisely the kind of judgement interviewers look for.

7.3 Croston's method and the SBA correction

Croston (1972) observed that for intermittent demand it is better to forecast two things separately: the **size** of a demand when it occurs (z) and the **interval** between demands (p). Each is smoothed with exponential smoothing, and the forecast of demand per period is their ratio:

$$\hat{y} = \hat{z} / \hat{p} \qquad (\text{SBA: } \hat{y} = (1 - \alpha/2) \cdot \hat{z} / \hat{p})$$

Syntetos and Boylan (2005) proved Croston's estimator is biased upward and introduced the correction factor $(1 - \alpha/2)$: this is `CrostonSBA`.

7.4 ADIDA: temporal aggregation

ADIDA (Aggregate-Disaggregate Intermittent Demand Approach) sums daily demand into larger buckets (e.g. weekly totals, where zeros mostly disappear), forecasts the smoother aggregate series with exponential smoothing, then divides back to daily. Aggregation is a simple, powerful trick for noisy data.

> **INTERVIEW ANGLE**
>
> "Why not Prophet?" Prophet is a fine baseline (trend + Fourier seasonality + holidays), but it fits one Stan model per series and is slow at 800+ products; on M-competition style data it is usually beaten by ETS/ARIMA. AutoARIMA with weekly seasonality already gives us a SARIMA baseline. Adding Prophet is listed as an exercise.

LECTURE 8

Machine learning: a global LightGBM model

8.1 Local versus global models

Statistical models are **local**: one model per product, which only learns from 2 years of one series. A **global** model is a single model trained on all 823 products at once. It can learn patterns shared across products ("SNAP lifts food sales", "demand dips when the price rises") and borrow strength from data-rich products for sparse ones. Global gradient boosting models won the M5 accuracy competition.

8.2 Forecasting as supervised regression

A tree model has no notion of time. We turn forecasting into a regression problem: each row is (product, day); the target is that day's sales; the features describe what was knowable beforehand. Features in `features.py`:

Group	Features	Captures
Lags	`lag_28, lag_35, lag_42, lag_49, lag_56, lag_364`	Recent level; same weekday in past weeks; same day last year.
Rolling stats	mean of last 7/28/56/112 days, std of 28, share of zero days in 28 (all ending 28 days ago)	Level, volatility, intermittency.
Weekday profile	`dow_mean_4w`: average of the same weekday over 4 weeks	The product's own weekly shape.
Calendar	day of week, day of month, month, week of year	Seasonality, pay-day effects.
Identity	`unique_id` as a categorical	Product-specific baseline.
Events & SNAP	event name/type, is_event, days to next event, snap, is_closed	Holidays, stock-up before events, benefit days.
Price	price, price vs. own historical mean, price vs. own historical max (discount), week-over-week change, price vs. department	Promotions and price elasticity.

8.3 Direct versus recursive multi-step forecasting

To forecast 28 days ahead with a `lag_1` feature, you would need tomorrow's sales to predict the day after: you must feed predictions back in (**recursive** strategy), and errors compound. Our alternative: use only lags ≥ 28. Then every feature for every day in the 28-day window is already known at the cutoff, and one model predicts all 28 days in a single pass (**direct** strategy). The price is that we lose the very freshest information (lags 1–27). A common upgrade is one model per horizon bucket (e.g. days 1–7 use lag 7).

8.4 Gradient boosting in one paragraph

Gradient boosting builds an ensemble of shallow decision trees *sequentially*. Each new tree is fitted to the gradient of the loss with respect to the current predictions (for squared loss: the residuals), so each tree corrects the mistakes of the ensemble so far. A small learning rate (0.05) shrinks each tree's contribution, trading more trees for better generalisation. LightGBM makes this fast with histogram-based splits and leaf-wise growth, and handles categorical features and missing values natively.

8.5 Tweedie loss for sales counts

Sales are non-negative, often exactly zero, and right-skewed. Squared loss assumes symmetric noise and can predict negatives. The **Tweedie** distribution with power $1 < p < 2$ is a compound Poisson-Gamma: a Poisson number of purchases, each with a Gamma-distributed size. It has a point mass at zero and a continuous positive part: exactly the shape of retail demand. We use $p = 1.1$ (close to Poisson), the setting popular among top M5 solutions.

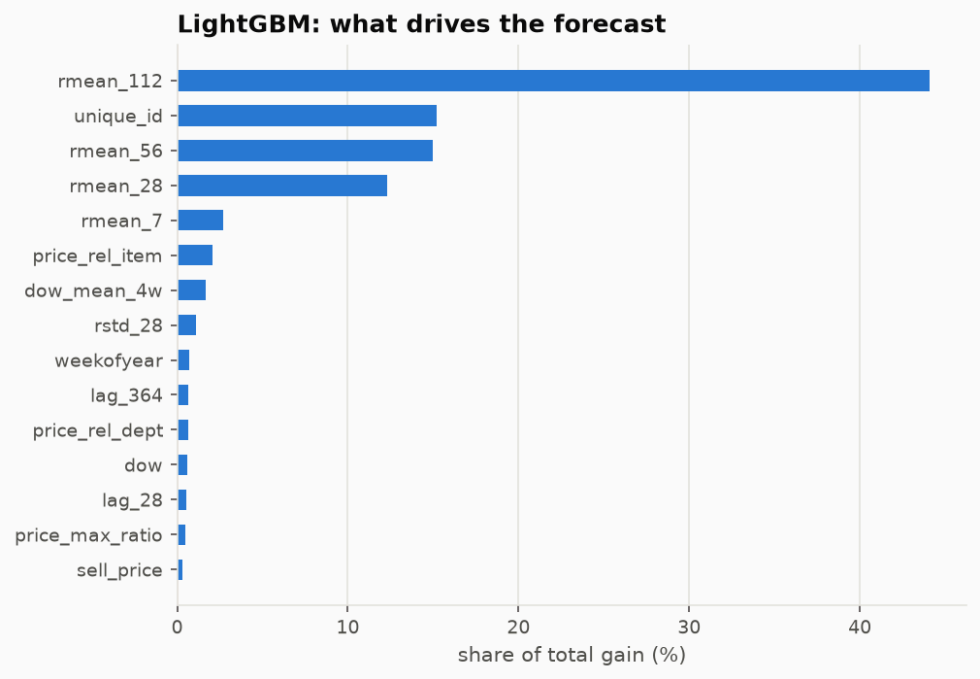


Figure 8.1. Feature importance (share of total split gain) of the LightGBM model in the final fold. Long and recent rolling means dominate (the product's level), followed by product identity; the price relative to the product's usual price is the strongest external signal.

COMMON MISTAKE: EARLY STOPPING ON THE TEST FOLD

Using the test window to choose the number of trees (early stopping) leaks the answer into the model. We use a fixed number of rounds (700). A proper alternative is an inner validation window carved from the training data.

LECTURE 9

Demand signal enrichment: is external data worth it?

Business forecasting is rarely about the sales history alone. Retailers know their promotion plan, the holiday calendar, and (in the U.S.) benefit payment schedules. To *measure* the value of such information rather than assume it, we run an **ablation study**: two LightGBM models, identical except that one sees only sales history + calendar, and the other also sees events, SNAP and prices.

Model	WRMSSE	WAPE	Bias
LightGBM (sales history only)	0.819	58.4%	+1.5%
LightGBM (+ events, prices, SNAP)	0.793	54.9%	+3.3%

Adding events, SNAP and price features reduces WRMSSE by **3.3%** and WAPE by 3.5% points. The relative price (`price_rel_item`) is the strongest non-history feature in Figure 8.1, consistent with promotions driving food demand. Note that even the history-only LightGBM beats every per-series statistical model: the global learning itself is a large part of the gain.

KEY IDEA: ABLATION IS HOW YOU PROVE A FEATURE MATTERS

Feature importance tells you what a model *used*; an ablation tells you what it is *worth*. Remove one group, retrain, re-evaluate on the same folds. The same design extends to weather, local events or Google Trends: add the signal, run the ablation, keep it only if it pays for its data-engineering cost.

PROFESSOR'S NOTE: "KNOWN IN ADVANCE" IS AN ASSUMPTION

Using future prices is legitimate only if prices are decided before the forecast is made. For the retailer's own price plan this is true. For a competitor's price, or for weather, it is not: you would need a *forecast* of that driver, with its own error. Always state this assumption explicitly.

LECTURE 10

Ensembles: why combining forecasts works

10.1 The evidence

Bates and Granger showed in 1969 that a combination of two forecasts can beat both. Half a century of M-competitions confirmed it: in M4 (2018) the top methods were combinations or hybrids of statistical and ML models, and in M5 (2020) the winners blended many gradient boosting models. Modern production systems therefore orchestrate *several* models rather than betting on one. Open-source tools such as TimeCopilot take this further, running hundreds of candidate models (statistical, ML and pretrained foundation models) behind an LLM layer that selects and explains them; our project is a small, transparent version of the same idea.

10.2 Why it works: a little algebra

Average two unbiased forecasts with errors e_1, e_2 (variance σ^2 each, correlation ρ). The error variance of the average is

$$\mathrm{Var}\left((e_1 + e_2) / 2 \right) = \sigma^2 \, (1 + \rho) / 2$$

Unless the errors are perfectly correlated ($\rho = 1$), the combination has lower variance than either model. The less correlated the members, the bigger the gain. That is why we combine models of *different families*: ETS (smoothing), ARIMA (autocorrelation) and LightGBM (cross-sectional learning with external drivers) make different kinds of mistakes.

10.3 Three combiners (`ensemble.py`)

- **Mean**: equal weights. Famous "forecast combination puzzle": it is surprisingly hard to beat, because estimated weights add estimation error.
- **Median**: robust if one member fails badly on some product.
- **Weighted**: weights $\propto$ 1 / WRMSSE, estimated *only on earlier folds*. Fold 1 has no past and uses equal weights.

> **COMMON MISTAKE: FITTING ENSEMBLE WEIGHTS ON THE TEST SET**
>
> If you choose weights by looking at which model did best on the same window you report, the ensemble's score is optimistically biased. Our weighted ensemble for fold k only uses scores from folds $1 \ldots k-1$.

LECTURE 11

Results of the model comparison

All models, 3 rolling folds of 28 days, metrics averaged across folds. Lower WRMSSE and WAPE are better; bias close to 0 is better; skill is the improvement over the seasonal naive.

Model	WRMSSE	WAPE	Bias	Skill vs. seasonal naive
Ensemble (weighted)	**0.784**	**54.6%**	**+3.1%**	**23.7%**
Ensemble (mean)	0.784	54.6%	+3.1%	23.7%
LightGBM (+ events, prices, SNAP)	0.793	54.9%	+3.3%	22.9%
Ensemble (median)	0.802	55.6%	+3.5%	22.0%
LightGBM (sales history only)	0.819	58.4%	+1.5%	20.3%
AutoETS	0.830	57.2%	+3.3%	19.3%
AutoARIMA	0.836	57.4%	+2.8%	18.7%
CrostonSBA	0.846	59.4%	+0.7%	17.8%
ADIDA	0.855	58.3%	+5.5%	16.9%
Moving average (28d)	0.866	57.9%	+2.4%	15.7%
Seasonal naive (last week)	1.028	68.0%	+1.1%	0.0%
Naive	1.210	80.4%	+23.3%	-17.7%

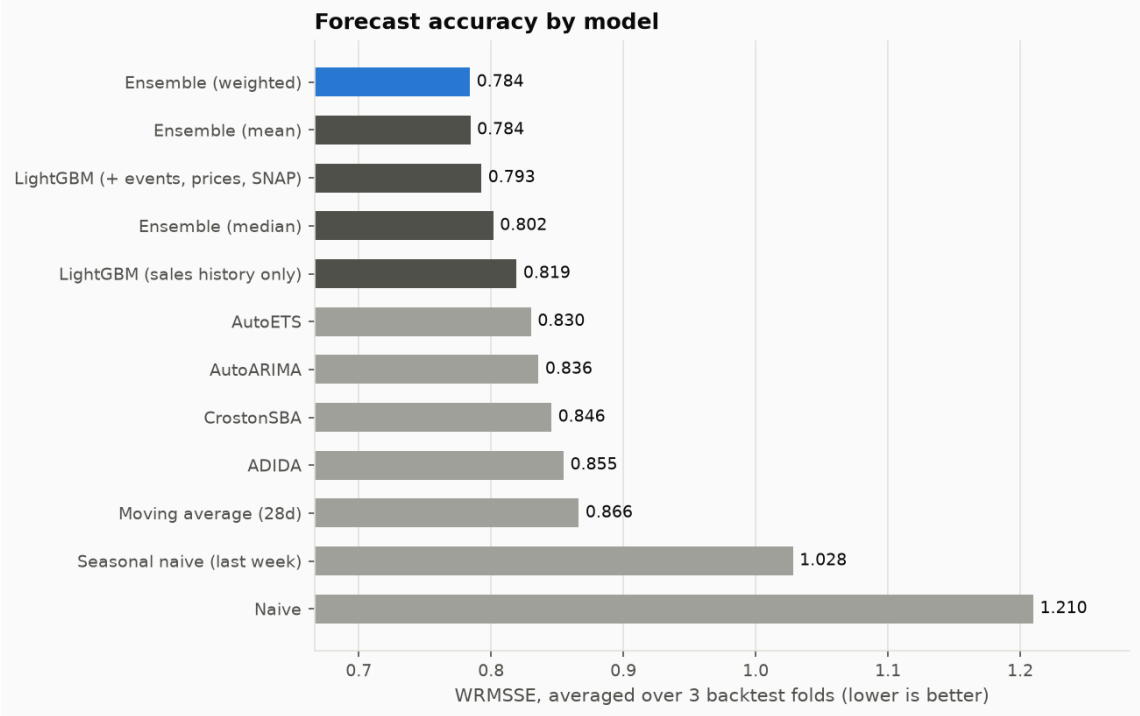


Figure 11.1. WRMSSE by model (lower is better). Grey: benchmarks and per-series statistical models; dark: LightGBM variants and ensembles; blue: best.

How to read these results

1. The ensemble wins. Ensemble (weighted) has the lowest WRMSSE (0.784), 23.7% better than the seasonal naive and 1.1% better than its best single member (LightGBM, 0.793). This reproduces, on a small scale, the central finding of M4 and M5.

2. Global ML beats local statistics. LightGBM (0.793) beats the best per-series model (AutoETS, 0.830). Learning across 823 products and using prices and events is worth more than fitting each product in isolation.

3. Smoothing beats copying. The 28-day moving average (0.866) beats the seasonal naive (1.028) even though the latter models the weekly cycle. With 37.1% zero days, copying last week copies its noise; averaging cancels it. This is the intermittency lesson of Lecture 4 in numbers.

4. Bias is small everywhere (between +0.7% and +5.5%), except the naive model which over-forecasts by +23%. Croston-SBA has the lowest bias (+0.7%), as its correction factor intends. Note also that WAPE stays above 50% for every model: daily SKU-level demand is genuinely noisy, which is exactly why safety stock exists.

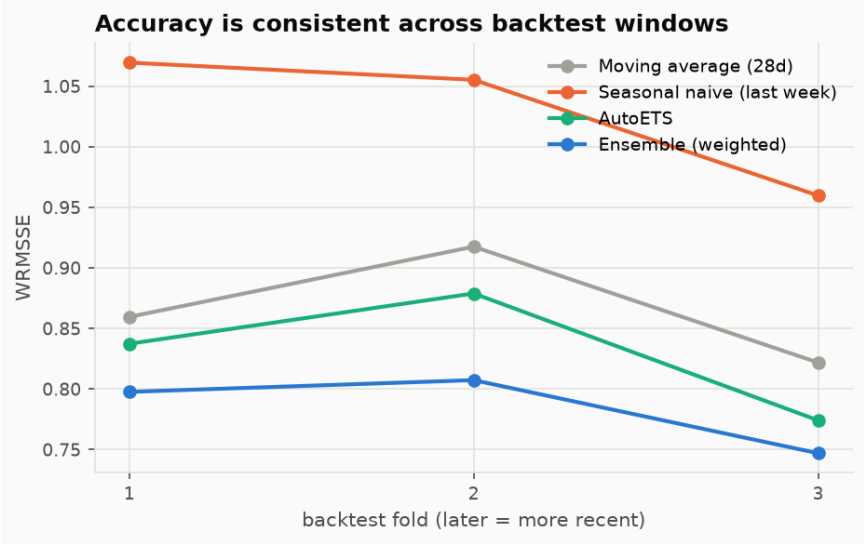


Figure 11.2. WRMSSE per fold. A ranking that holds across all folds is trustworthy; one that flips is noise.

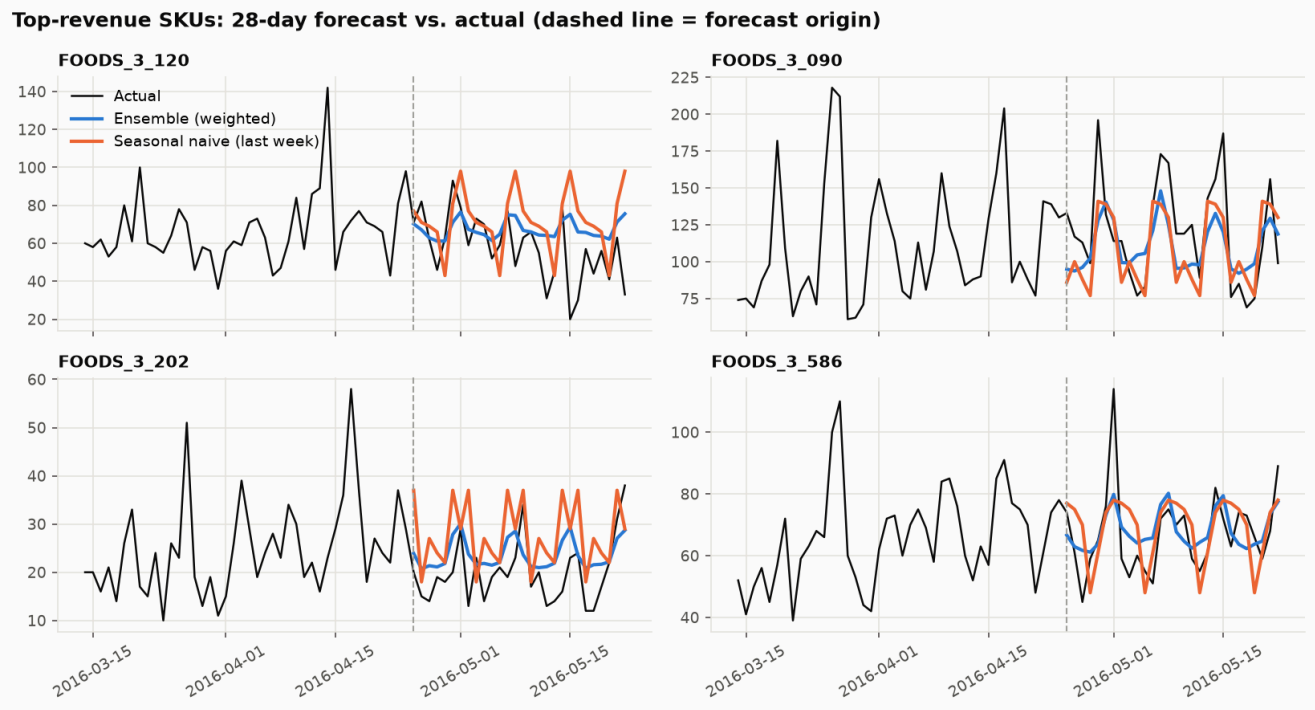


Figure 11.3. The four highest-revenue products in the final fold: actual sales (black), the best model (blue) and the seasonal naive (orange). Note how the best model forecasts a smooth expected value: it does not try to predict individual spikes, which are unpredictable noise. The spikes are what safety stock is for (next lecture).

LECTURE 12

From forecast to decision: inventory optimization

12.1 Two kinds of stock

Cycle stock covers the *expected* demand until the next delivery: that is the forecast. **Safety stock** covers the *uncertainty* around it: that is the forecast error. A better forecast reduces the error, which reduces safety stock, which frees cash. That sentence is the whole business case of this project.

12.2 Service level, z and safety stock

Assume daily forecast errors are independent with standard deviation σ. Over a protection period of L days, the errors add up to a standard deviation of $\sigma\sqrt{L}$ (variances add). To be covered with probability a (the **cycle service level**), hold

$$\mathrm{SS} = z_{\alpha} \cdot \sigma \cdot \sqrt{L} \qquad \mathrm{ROP} = \bar{d} \cdot L + \mathrm{SS}$$

where z_{α} is the standard normal quantile ($z = 1.28$ for 90%, 1.645 for 95%, 2.33 for 99%) and ROP is the **reorder point** of a continuous-review system: when stock falls to ROP, order. Note how fast z grows: going from 95% to 99% raises safety stock by 42%. Perfect availability is extremely expensive.

> **KEY IDEA: Σ IS THE FORECAST ERROR, NOT THE DEMAND VOLATILITY**
>
> Textbooks often plug in the standard deviation of demand. That is only correct if your forecast is the long-run average. If you forecast well, the uncertainty that remains is the *forecast error*, which is smaller. We estimate σ per product from its out-of-sample backtest residuals: honest, and it rewards better models with less safety stock.

12.3 The policy we simulate: periodic review (R, S)

Stores reorder on a schedule, so we use a periodic-review, order-up-to policy. Every $R = 7$ days we look at the **inventory position** (on hand + on order) and order enough to raise it to the level S. The order arrives after a lead time $L = 3$ days, so each order must protect the shelf for $R + L$ days:

$$S_t = \Sigma_{k=t}^{t+R+L-1} \hat{y}_k + z \cdot \sigma \cdot \sqrt{(R + L)} \qquad \mathrm{Order}_t = \max(0, S_t - \mathrm{position}_t)$$

Unmet demand is lost (customers do not wait for groceries). The simulation in `inventory.py` replays the final backtest fold day by day with real sales, for every product at once (vectorised with NumPy), for several models and service levels.

12.4 Two service measures

- **Cycle service level (CSL)**: probability of no stockout during a cycle. It is what z targets.
- **Fill rate**: share of demand actually served from stock. It is what customers feel, and what we report.

They differ: a 90% CSL usually yields a fill rate well above 90%, because when a stockout happens it is typically short.

12.5 Costs

Holding cost = unit value × 25% per year (capital, storage, shrinkage), charged per unit-day on the shelf. **Lost-sale cost** = lost units × price × 30% gross margin. Raising the service level trades the second for the first.

12.6 Results: the efficient frontier

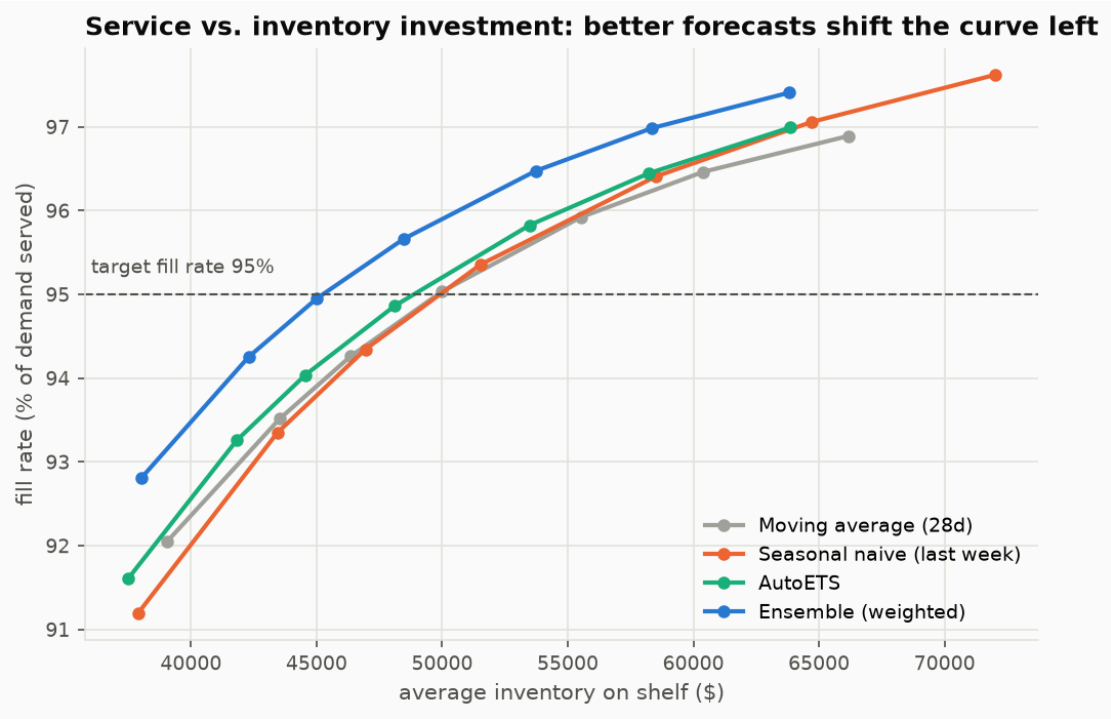


Figure 12.1. Each point is one simulated service-level target. A curve further to the left reaches the same fill rate with less money on the shelf.

Forecast driving the policy	Avg. inventory ($) for 95% fill rate	vs. moving average
Ensemble (weighted)	$45,247	-9.2%
AutoETS	$48,870	-1.9%
Moving average (28d)	$49,830	—
Seasonal naive (last week)	$49,912	+0.2%

To serve 95% of demand, the policy driven by Ensemble (weighted) needs on average **$45,247** of stock on the shelf for this one department of one store, versus $49,830 with a 28-day moving average: **9.2% less inventory for the same service**. Scaled to all departments and stores of a retailer, that is the money a better forecast frees.

> **PROFESSOR'S NOTE: LIMITS OF THIS SIMULATION**
>
> One 28-day window, no minimum order quantities, case packs, shelf capacity, or perishability, and $\sigma\sqrt{L}$ assumes independent daily errors (real errors are positively correlated, so the true risk is a bit higher). These are honest caveats to state, and natural extensions to add.

LECTURE 13

Deployment: the production forecast and the inventory plan

A backtest proves the method; it does not tell a store manager what to order on Monday. The `forecast` stage retrains the winning approach (Ensemble (weighted)) on *all* history and forecasts the 28 days that follow it (2016-05-23 to 2016-06-19). M5 provides the calendar and planned prices for those days, just as a retailer knows its own promotion calendar.

The output `reports/inventory_plan.csv` gives, for each product: forecast for the next 7 and 28 days, daily σ from the backtest, safety stock, reorder point, order-up-to level and the dollar value of the safety stock. Totals for this run: **81,739** units forecast over 28 days; safety stock worth **$26,399** at a 95% service target.

Product	Next 7d	σ/day	Safety stock	Reorder point	Order-up-to	SS value ($)
FOODS_3_120	355.6	24.60	127.9	230.7	641.3	637.12
FOODS_3_362	164.1	25.22	131.2	144.4	362.5	335.86
FOODS_3_499	150.4	25.09	130.5	142.2	348.7	334.06
FOODS_3_282	208.3	21.54	112.0	156.0	410.3	286.77
FOODS_3_090	691.8	33.35	173.5	411.0	1145.7	277.59
FOODS_3_444	112.4	6.84	35.6	78.4	201.5	248.19
FOODS_3_382	4.8	20.52	106.7	62.9	115.0	211.31
FOODS_3_233	131.8	15.51	80.7	99.2	266.2	206.53

Top rows of the plan, sorted by the dollar value of safety stock: these are the products where better forecasting frees the most cash.

INTERVIEW ANGLE: "HOW WOULD YOU PUT THIS IN PRODUCTION?"

- Schedule `demandcast all` weekly (cron, Airflow, or a cloud scheduler) after the sales data lands.
- Write forecasts and the plan to a database table the replenishment system reads.
- Monitor: track WRMSSE and bias of last week's forecast every week; alert on drift.
- Keep the benchmark running in parallel as a safety net, and retrain on a fixed cadence.

LECTURE 14

Engineering the project: structure, tests, CI, reproducibility

Recruiters open your repository for about 60 seconds. A clean structure and a README with results signal that you can work in a team codebase, not only in notebooks.

```
retail-demand-forecasting/
├── configs/default.yaml        # every tunable setting in one place
├── src/demandcast/
│   ├── data.py                 # download, clean, validate, demand classes, future frame
│   ├── features.py             # leak-free lag / rolling / calendar / price features
│   ├── models.py               # benchmarks, statistical models, global LightGBM
│   ├── ensemble.py             # mean / median / leak-free weighted combination
│   ├── evaluation.py           # WRMSSE, WAPE, bias
│   ├── backtest.py             # rolling-origin evaluation loop
│   ├── inventory.py            # safety stock, ROP, (R,S) simulation, plan
│   ├── plots.py                # all figures
│   └── pipeline.py             # CLI: prepare|eda|backtest|inventory|forecast|all
├── tests/                      # 11 unit tests on synthetic data (no download needed)
├── reports/                    # metrics CSVs + figures produced by the pipeline
├── .github/workflows/ci.yml    # lint + tests on every push
├── Makefile, pyproject.toml, requirements.txt
```

Principles applied

- **Package, not notebook.** Code in `src/` is importable and testable; notebooks (if any) only call it.
- **Configuration over code.** Store, department, horizon, folds, model and inventory parameters live in YAML.
- **One command reproduces everything.** `make all` downloads the data and regenerates every number and figure.
- **Tests that test the science.** Not only "does it run": the leakage test, a perfect forecast scoring exactly zero, textbook safety-stock values, and monotonic fill rate in z.
- **Continuous integration.** GitHub Actions runs ruff (lint) and pytest on every push: the green badge tells a reviewer the code works.
- **Pinned environment.** `requirements.txt` records the exact versions behind the reported results.

LECTURE 15

Presenting the project in an interview

15.1 The 60-second pitch (STAR)

> **TEMPLATE, FILL WITH YOUR OWN WORDS**
>
> **Situation.** Retailers lose money both to empty shelves and to excess stock; the lever is the demand forecast. **Task.** Forecast 28 days of daily demand for 823 Walmart food products and turn it into stocking decisions. **Action.** I cleaned the M5 data, built a rolling-origin backtest, compared benchmarks, ETS, SARIMA, Croston-type models and a global LightGBM with price and event features, then ensembled them and simulated an inventory policy. **Result.** The ensemble cut forecast error by 24% versus a seasonal naive benchmark across three backtest windows, price and event data alone added 3%, and in simulation the forecast-driven policy held 9% less inventory than a moving-average policy at a 95% fill rate.

15.2 Questions you should expect, with model answers

How did you prevent data leakage?

Time-ordered rolling-origin folds; all sales features lagged ≥ the 28-day horizon; ensemble weights estimated only on earlier folds; fixed boosting rounds instead of early stopping on test data; and a unit test that tampers with future sales and asserts features do not change.

Why WRMSSE instead of MAPE or accuracy?

Many zero days make MAPE undefined. WRMSSE is scale-free, targets the mean (useful for inventory sums) and weights products by revenue. I also report WAPE and bias for business readers.

Why did LightGBM beat ARIMA here?

It is global: it learns from 800+ products at once, uses prices, events and SNAP, and the Tweedie loss fits zero-inflated counts. ARIMA sees one short noisy series and no drivers.

Why does an ensemble help?

Errors of different model families are imperfectly correlated, so averaging reduces error variance: Var = $\sigma^2(1+\rho)/2$ for two members. M4 and M5 confirmed this empirically.

How do you get from a forecast to an order quantity?

Order-up-to level = forecast over review period + lead time + $z \cdot \sigma \cdot \sqrt{(R+L)}$, with σ the out-of-sample forecast error; order the difference to the inventory position.

What would you do with more time?

Probabilistic (quantile) forecasts to set stock directly at the required quantile; hierarchical reconciliation so SKU forecasts add up to store totals; horizon-specific models using lags shorter than 28; and pretrained foundation models (Chronos, TimesFM) as extra ensemble members.

What was the hardest part?

Pick a real one, e.g. AutoARIMA being 10× too slow and profiling it, or discovering that pre-launch zeros biased the models.

DO NOT

Do not claim results you cannot reproduce live, and do not present code you cannot explain line by line. Before any interview, rerun the pipeline yourself and change one parameter to see what happens: that is what turns this into *your* project.

LECTURE 16

Exercises and extensions

Ordered from quick to ambitious. Each one is a good commit (and a good interview story).

1. **Another slice.** Change `store_id` to `TX_2` or `dept_id` to `HOBBIES_1`. Does the ranking of models change? Why might hobby products behave differently from food?
2. **Horizon sensitivity.** Compute WRMSSE separately for days 1–7, 8–14, 15–21, 22–28. How fast does accuracy decay?
3. **Accuracy by demand class.** Join `reports/demand_classes.csv` to the forecasts. Do Croston/ADIDA win on lumpy products? Build a "class-aware" model selector.
4. **Prophet baseline.** Add Prophet with weekly + yearly seasonality and U.S. holidays to `models.py` and compare speed and accuracy.
5. **Shorter lags.** Train separate LightGBM models for horizons 1–7 (allowing lag 7) and 8–28. How much does fresh information help?
6. **Hyper-parameter tuning.** Use the first two folds to tune num_leaves and learning rate (e.g. with Optuna) and evaluate on fold 3 only.
7. **Probabilistic forecasts.** Train LightGBM with the quantile loss at 0.5 / 0.9 / 0.95, and set order-up-to levels from the quantile directly instead of $z \cdot \sigma$. Compare the inventory frontier.
8. **External data.** Add daily weather for the store's city (e.g. from the free Open-Meteo archive API) and run the enrichment ablation of Lecture 9.
9. **Foundation models.** Add a zero-shot pretrained model (Amazon Chronos, Google TimesFM, or via the TimeCopilot library) as an ensemble member.
10. **Hierarchical forecasting.** Forecast product, department and store levels and reconcile them (MinT) so they add up.
11. **Dashboard.** A small Streamlit app: pick a product, see history, forecast, and its safety stock.

Self-check quiz

1. Why can a random 80/20 split not be used to evaluate a forecasting model?
2. A product sells on 1 day in 4, always 3–4 units. Which Syntetos-Boylan class is it?
3. What does an RMSSE of 0.8 mean?
4. Why must sales features be lagged by at least 28 days in our design?
5. Two forecasts have equal error variance and correlation 0.5. By how much does their average reduce error variance?
6. Lead time 4 days, $\sigma = 3$ units/day, 95% CSL. What is the safety stock?
7. Why is the fill rate usually higher than the cycle service level?
8. What is the Tweedie loss and why is it suitable here?

Answers. 1: the model would train on the future of the test days (leakage). 2: intermittent (ADI $= 4 \geq 1.32$, low CV^2). 3: its error is 20% below the in-sample error of a one-step naive forecast. 4: so every feature in the 28-day window is known at the cutoff (direct strategy, no leakage). 5: variance becomes $\sigma^2(1+0.5)/2 = 0.75\sigma^2$, a 25% reduction. 6: $1.645 \times 3 \times \sqrt{4} \approx 9.9$ units. 7: stockouts, when they happen, typically affect only part of the cycle's demand. 8: the loss of a compound Poisson-Gamma distribution: it handles exact zeros and skewed positive counts.

LECTURE 17

Glossary and further reading

ADI Average inter-demand interval: periods per non-zero demand.

AIC Akaike Information Criterion: fit penalised by number of parameters, used to select models.

Backtest Evaluating a forecasting method on historical windows as if it had been used then.

Bias Systematic over- or under-forecasting: $\Sigma(\hat{y} - y)/\Sigma y$.

Cutoff / origin Last day of data available when the forecast is made.

CRISP-DM Six-phase process model for data projects.

Cycle service level Probability of no stockout during a replenishment cycle.

CV² Squared coefficient of variation of non-zero demand sizes.

Direct strategy Forecasting all horizons from features known at the cutoff, without feeding predictions back.

ETS Error-Trend-Seasonality state-space exponential smoothing.

Fill rate Share of demand served from stock.

Global model One model trained across many series.

Horizon How far ahead we forecast (28 days).

Intermittent demand Demand with many zero periods.

Inventory position On-hand stock plus stock on order.

Lead time Delay between placing and receiving an order.

Leakage Using information in training that would not be available at prediction time.

Panel data Many time series stacked in long format.

Reorder point Stock level that triggers a new order: $\bar{d} \cdot L + SS$.

RMSSE RMSE scaled by in-sample naive error.

Safety stock Extra stock covering forecast error: $z \cdot \sigma \cdot \sqrt{L}$.

SARIMA ARIMA with seasonal AR/I/MA terms.

SKU Stock-keeping unit: one distinct product.

SNAP U.S. food-assistance benefit paid on fixed days per state.

STL Seasonal-Trend decomposition using LOESS.

Tweedie loss Loss for compound Poisson-Gamma targets: zeros plus skewed positives.

WAPE Weighted absolute percentage error: $\Sigma|e|/\Sigma y$.

WRMSSE Revenue-weighted RMSSE, the M5 metric.

Further reading

- Hyndman, R.J. & Athanasopoulos, G. *Forecasting: Principles and Practice*, 3rd ed. (free online at otexts.com/fpp3). The single best textbook; chapters on ETS, ARIMA, evaluation and STL.
- Makridakis, S., Spiliotis, E. & Assimakopoulos, V. (2022). "M5 accuracy competition: Results, findings, and conclusions." *International Journal of Forecasting*, 38(4).
- Makridakis, S. et al. (2020). "The M4 Competition: 100,000 time series and 61 forecasting methods." *IJF*, 36(1).
- Syntetos, A.A., Boylan, J.E. & Croston, J.D. (2005). "On the categorization of demand patterns." *Journal of the Operational Research Society*, 56.
- Cleveland, R.B. et al. (1990). "STL: A seasonal-trend decomposition procedure based on Loess." *Journal of Official Statistics*, 6(1).
- Bates, J.M. & Granger, C.W.J. (1969). "The combination of forecasts." *Operational Research Quarterly*, 20(4).
- Ke, G. et al. (2017). "LightGBM: A highly efficient gradient boosting decision tree." *NeurIPS*.
- Silver, E.A., Pyke, D.F. & Thomas, D.J. *Inventory and Production Management in Supply Chains*, 4th ed. CRC Press. Safety stock and (R,S) policies.
- Chapman, P. et al. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*.
- Nixtla documentation for `statsforecast`; LightGBM documentation; TimeCopilot (timecopilot.dev) for a production-grade, multi-model forecasting architecture.